

10: GPU Operator & Kubebuilder Deep Dive

A Kubernetes node with an NVIDIA GPU cannot run GPU workloads until several pieces of software are installed and configured on it. The NVIDIA GPU Operator automates that work. It is also one of the largest operators in common use, and its design shows how an operator coordinates many node-level components, handles partial failure, and reports status.

This chapter explains what each GPU Operator component does, how a GPU node and a GPU Pod move through the system, and how GPUs are shared. It then lists the design practices that apply to any operator you build with Kubebuilder, and ends with a troubleshooting guide and interview questions.

Note: The GPU Operator changes quickly. Current releases are installed with a Helm chart and configured through a ClusterPolicy custom resource, usually named `cluster-policy`. Component names, labels, and configuration fields change between releases. The architecture described here is stable, but check exact names and flags against the documentation for the release you use.

This chapter covers

- The node-level software a GPU node needs, and why an operator manages it
- The GPU Operator's components: NFD, the driver, the container toolkit, the device plugin, GPU Feature Discovery, the MIG manager, DCGM, and the validator
- The device plugin API and the path from a Pod's GPU request to a device in the container
- Whole GPUs, MIG, time-slicing, MPS, and vGPU
- Operator design practices demonstrated by the GPU Operator
- Troubleshooting GPU allocation and GPU access in containers

Contents

1. The problem the GPU Operator solves.....	3
2. Installation and the reconciliation model.....	4
3. The components	5
3.1 The operator controller	5
3.2 Node Feature Discovery (NFD)	5
3.3 The driver	6
3.4 The container toolkit.....	6
3.5 The device plugin.....	6

3.6 GPU Feature Discovery (GFD)	7
3.7 The MIG manager	8
3.8 DCGM and the DCGM exporter	8
3.9 The operator validator	9
4. The lifecycle of a GPU node	10
5. From a Pod's GPU request to a device	11
6. Ways to share GPUs	12
7. Operator design practices from the GPU Operator	13
8. Troubleshooting	14
nvidia.com/gpu is missing from the node's allocatable resources.....	14
A Pod is Pending with 0/N nodes are available.....	14
The Pod runs, but nvidia-smi fails inside the container.....	14
GPU hardware errors.....	15
9. Interview questions	16
How does a Pod get a GPU?.....	16
What does the device plugin do?	16
What is the difference between the device plugin and the container toolkit?	16
How are drivers upgraded without disrupting workloads?	16
Why are most components DaemonSets?.....	16
How would you apply the GPU Operator's design to another operator?.....	16
Summary	17

1. The problem the GPU Operator solves

Before Kubernetes can schedule GPU workloads on a node, all of the following must be true:

1. The NVIDIA kernel driver is installed and loaded on the host.
2. The driver's user-space libraries, such as `libcuda.so`, can be made available to containers.
3. The container runtime is configured to add GPU devices and libraries to containers.
4. A device plugin advertises `nvidia.com/gpu` to the kubelet, so the node reports GPU capacity.
5. The node has labels that describe its GPUs, so workloads can select a GPU model or MIG configuration.
6. GPU metrics are exported for monitoring.
7. A test workload has confirmed that the whole stack works.

Doing this by hand on every node is slow, and it has to be repeated when nodes are added, reimaged, or upgraded. The GPU Operator turns the requirements into declarative cluster state: when a GPU node joins the cluster, the operator deploys and configures each component on it automatically.

2. Installation and the reconciliation model

Install the operator with Helm:

```
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update
```

```
helm install gpu-operator nvidia/gpu-operator \
  --namespace gpu-operator \
  --create-namespace
```

The chart installs the gpu-operator namespace, the CRDs (including ClusterPolicy), the operator's controller Deployment, its service accounts and RBAC, and a default ClusterPolicy:

```
kubectl get clusterpolicy
NAME          AGE
cluster-policy 2m
```

The ClusterPolicy object is the desired state for GPU software across the cluster. The operator's controller, built on controller-runtime, watches the ClusterPolicy, the cluster's nodes, and the resources it creates, and reconciles the node-level components to match. This is the model described in 04-kubernetes-operator.md: a custom resource is the API, and a controller turns it into built-in Kubernetes objects.

```
User
| helm install, or kubectl edit clusterpolicy
▼
ClusterPolicy (desired state)
|
▼
GPU Operator controller (controller-runtime manager)
| reads the ClusterPolicy, node labels, and component status
| watches the DaemonSets and other objects it owns
▼
DaemonSets, Deployments, ConfigMaps, and RuntimeClass
|
├─ Node Feature Discovery → labels nodes that have NVIDIA PCI devices
├─ Driver → loads the kernel driver, provides user-space libraries
├─ Container Toolkit → configures the container runtime and CDI
├─ Device Plugin → advertises nvidia.com/gpu to the kubelet
├─ GPU Feature Discovery → labels nodes with GPU model, memory, and MIG details
├─ MIG Manager → applies MIG partition layouts
├─ DCGM Exporter → exports GPU metrics to Prometheus
└─ Operator Validator → confirms each layer works
```

3. The components

3.1 The operator controller

The controller has these responsibilities:

- Reconcile the `ClusterPolicy`.
- Create and update a `DaemonSet` or other objects for each enabled component.
- React when nodes are added, removed, or relabeled.
- Enable or disable components according to fields such as `driver.enabled`, `toolkit.enabled`, and `devicePlugin.enabled`. A cluster whose nodes already have a driver installed, for example, sets `driver.enabled: false`.
- Order the components, because each one depends on the ones before it.
- Report the state of each component in the `ClusterPolicy` status.

The controller-runtime patterns it uses:

Pattern	How the GPU Operator uses it
One manager with several watches	Watches the <code>ClusterPolicy</code> , nodes, and owned <code>DaemonSets</code> , and maps each event to a reconcile
Declarative enablement	A boolean field in the spec controls whether a component's objects exist
Status reports progress	The <code>ClusterPolicy</code> status shows whether the stack is ready or still <code>notReady</code>
Upgrades are rollouts	A new driver version is rolled out node by node, not replaced in place on all nodes at once

3.2 Node Feature Discovery (NFD)

NFD detects hardware and kernel features on each node and records them as node labels: PCI devices, CPU features, kernel modules, and more. The GPU Operator can deploy NFD itself, or use an existing installation.

The operator uses these labels to decide which nodes need GPU components. A node with an NVIDIA PCI device receives a label such as `feature.node.kubernetes.io/pci-10de.present=true` (10de is NVIDIA's PCI vendor ID), and the operator marks the node with `nvidia.com/gpu.present=true`. The component `DaemonSets` select nodes with that label, so CPU-only nodes do not run them.

Selecting nodes by discovered labels is a general technique for any operator that deploys per-node software only to certain hardware.

3.3 The driver

The driver component runs a container on each GPU node that installs and loads the NVIDIA kernel driver and provides the user-space libraries. Running the driver as a container lets the operator manage its version without building it into the node image, but it is complex:

- The kernel module must be built for, or match, the node's exact kernel version.
- The driver container must be privileged, because it loads kernel modules and writes to host paths such as `/run/nvidia`.
- Loading a new driver requires that no process is using the GPU, so running GPU workloads must be stopped first.
- After installation, the node must be validated before GPU workloads are scheduled on it.

If the node image already contains the driver, as on many managed Kubernetes services, disable this component.

3.4 The container toolkit

The NVIDIA Container Toolkit makes the host's GPUs usable inside containers. The toolkit component:

- Installs the NVIDIA container runtime and configures containerd or CRI-O to use it.
- Creates a RuntimeClass named `nvidia` that selects that runtime handler.
- In current versions, generates **Container Device Interface (CDI)** specifications, such as `/var/run/cdi/nvidia.yaml`. A CDI specification lists, for each GPU, the device nodes, library mounts, and environment variables to add to a container, so any CDI-aware runtime can add GPUs without an NVIDIA-specific hook.

A workload that uses the runtime class specifies:

```
runtimeClassName: nvidia
resources:
  limits:
    nvidia.com/gpu: 1
```

Without CDI, the NVIDIA runtime calls `nvidia-container-cli` before the container starts. It reads `NVIDIA_VISIBLE_DEVICES` and mounts the corresponding device nodes and driver libraries into the container.

3.5 The device plugin

A device plugin is the Kubernetes extension point for hardware resources other than CPU and memory. The NVIDIA device plugin runs as a DaemonSet on GPU nodes and communicates with the kubelet over gRPC on a Unix socket.

The device plugin API has five methods:

Method	Purpose
GetDevicePluginOptions	Tells the kubelet which optional methods the plugin implements
ListAndWatch	Streams the list of devices, with their IDs and health, and sends an update whenever a device's health changes
GetPreferredAllocation	Optional. Suggests which specific devices to assign, for example GPUs connected by NVLink
Allocate	Called when a container is being created with devices assigned. Returns the environment variables, mounts, device nodes, or CDI device names for those devices
PreStartContainer	Optional. Performs device-specific setup before the container starts

Registration and allocation proceed as follows:

1. The plugin starts, opens its own gRPC socket, and registers with the kubelet through `/var/lib/kubelet/device-plugins/kubelet.sock`, giving its resource name, `nvidia.com/gpu`.
2. The kubelet connects back and calls `ListAndWatch`.
3. The kubelet sets the node's capacity and allocatable for `nvidia.com/gpu` to the number of healthy devices.
4. The scheduler places Pods by comparing their requests with each node's allocatable count. The scheduler sees only a number, not individual GPUs.
5. When the kubelet admits a Pod, its device manager chooses specific devices for each container and calls `Allocate` with their IDs.
6. The plugin returns the runtime settings for those devices, and the kubelet passes them to the container runtime.

The GPU Operator does not schedule Pods. It installs the device plugin, and the scheduler and kubelet use it to allocate GPUs.

If the kubelet restarts, it removes its socket and every plugin must register again. The NVIDIA plugin watches the socket directory and re-registers automatically.

3.6 GPU Feature Discovery (GFD)

GFD runs on each GPU node and adds labels that describe the GPUs:

Label	Example value
<code>nvidia.com/gpu.product</code>	NVIDIA-A100-SXM4-80GB
<code>nvidia.com/gpu.memory</code>	81920 (MiB)
<code>nvidia.com/gpu.count</code>	8
<code>nvidia.com/cuda.driver.major</code>	550
<code>nvidia.com/mig.capable</code>	true

Workloads use these labels to select hardware:

```
nodeSelector:
  nvidia.com/gpu.product: NVIDIA-A100
```

The label value must match exactly. In this example, the value would need to be NVIDIA-A100-SXM4-80GB to match the node in the table above. Autoscalers, billing systems, and scheduling policies also use these labels to make decisions based on the GPU model.

3.7 The MIG manager

Multi-Instance GPU (MIG) divides a supported GPU, such as an A100 or H100, into as many as seven GPU instances. Each instance has dedicated memory and compute and its own fault isolation. The MIG manager:

- Enables or disables MIG mode on each GPU.
- Creates the GPU instances described by a named layout, selected with the `nvidia.com/mig.config` node label.
- Stops the GPU clients on the node while it changes the layout, then restarts them.
- Reapplies the layout after a reboot.

The device plugin exposes MIG instances according to a *MIG strategy*:

Strategy	Resources advertised
none	Whole GPUs as <code>nvidia.com/gpu</code> ; MIG is not used
single	Every GPU on the node uses the same profile, and each MIG instance is advertised as <code>nvidia.com/gpu</code>
mixed	GPUs can use different profiles, and each profile has its own resource name, such as <code>nvidia.com/mig-1g.10gb</code> or <code>nvidia.com/mig-3g.40gb</code>

MIG isolation is enforced by the GPU hardware. Time-slicing, described in section 6, shares a GPU in software without memory isolation.

3.8 DCGM and the DCGM exporter

NVIDIA Data Center GPU Manager (DCGM) collects GPU telemetry and health information. The DCGM exporter runs as a DaemonSet and publishes the data as Prometheus metrics. Commonly used metrics include:

Metric	Meaning
DCGM_FI_DEV_GPU_UTIL	GPU utilization, in percent
DCGM_FI_DEV_MEM_COPY_UTIL	Memory bus utilization, in percent
DCGM_FI_DEV_FB_USED, DCGM_FI_DEV_FB_FREE	Framebuffer memory used and free, in MiB
DCGM_FI_DEV_GPU_TEMP	GPU temperature, in degrees Celsius
DCGM_FI_DEV_POWER_USAGE	Power draw, in watts
DCGM_FI_DEV_XID_ERRORS	The most recent Xid error code
DCGM_FI_DEV_ECC_SBE_VOL_TOTAL, DCGM_FI_DEV_ECC_DBE_VOL_TOTAL	Single-bit and double-bit ECC errors

The exporter adds Pod, namespace, and container labels to each metric, so usage can be attributed to workloads. Control plane monitoring, billing, and node health checks use these metrics.

3.9 The operator validator

The validator confirms that each layer works before workloads rely on it. It runs as a DaemonSet whose init containers check, in order, that the driver is loaded, that the container toolkit is configured, that a CUDA sample runs in a container, and that the device plugin advertises GPUs. Each successful step writes a status file that the next component waits for.

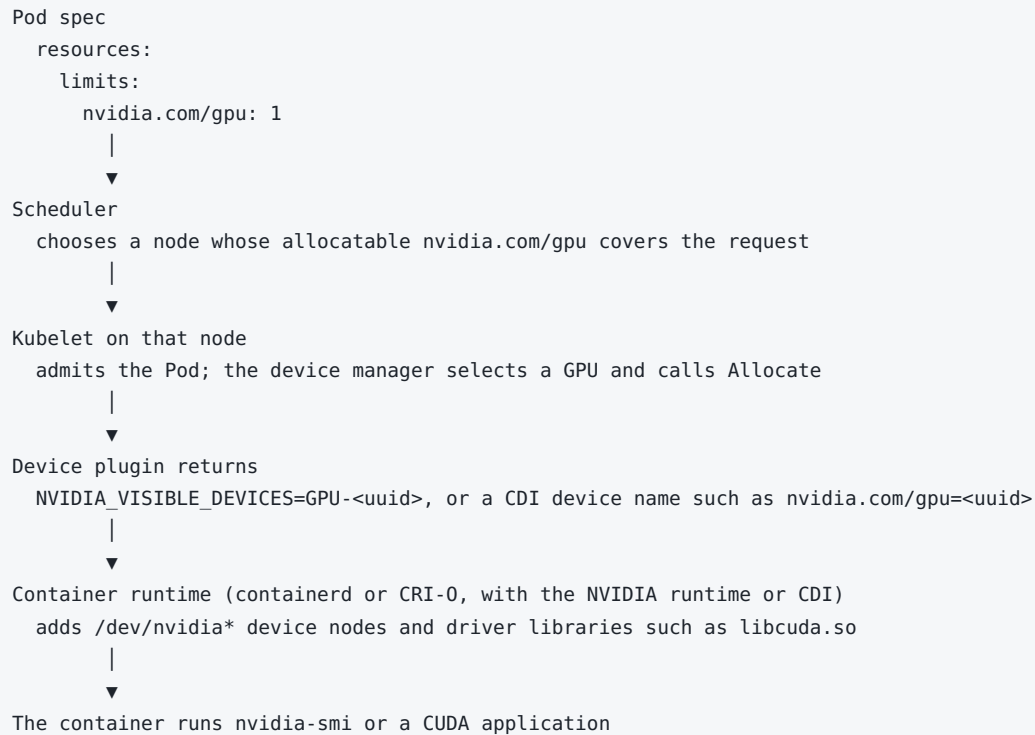
The validator demonstrates a general principle: a DaemonSet reporting its Pods as ready does not prove that the feature works. An operator can run a small workload that exercises the real path.

4. The lifecycle of a GPU node

1. A node with GPUs joins the cluster.
2. NFD labels the node with its PCI devices.
3. The GPU Operator marks the node as a GPU node, and the component DaemonSets schedule Pods on it.
4. The driver Pod builds or loads the kernel driver.
5. The toolkit Pod configures the container runtime, the `nvidia RuntimeClass`, and CDI.
6. The device plugin registers with the kubelet and advertises `nvidia.com/gpu`, or MIG resources.
7. GPU Feature Discovery adds labels for the GPU model, memory, and count.
8. The validator confirms the stack, and DCGM begins exporting metrics.
9. Users create Pods that request `nvidia.com/gpu`. The scheduler places them, the kubelet calls `Allocate`, and the runtime adds the devices to the containers.

When a node reboots, the driver must be loaded again, the MIG layout must be reapplied, and the device plugin must register again. Because each component is a DaemonSet Pod that repeats its setup when it starts, the node returns to a working state without manual steps.

5. From a Pod's GPU request to a device



nvidia.com/gpu is an extended resource, and Kubernetes requires extended resources to be specified in limits. If requests is omitted, it defaults to the limit; if it is specified, it must equal the limit. The Pod above is valid because the resource appears in limits.

6. Ways to share GPUs

Mode	Isolation	Typical use
Whole GPU (nvidia.com/gpu:1)	Complete: one workload owns the GPU	Training and large inference workloads
MIG	Hardware isolation of memory, compute, and faults	Several medium workloads with predictable performance
Time-slicing	None for memory or faults; workloads take turns on the GPU	Development, notebooks, and light inference where oversubscription is acceptable
MPS (Multi-Process Service)	Shared GPU context with configurable limits on compute share and memory; a fault can affect other clients	Many small inference processes that need higher throughput than time-slicing
vGPU	Virtual GPUs assigned to virtual machines by the hypervisor	GPU-accelerated VMs

- MIG does not add memory. Each instance receives a fixed part of the GPU's memory.
- With time-slicing, the device plugin advertises each physical GPU as several nvidia.com/gpu replicas. Workloads that share a GPU can run out of memory because of each other, and their latency varies with each other's load.

7. Operator design practices from the GPU Operator

These practices apply to any operator you build with Kubebuilder:

1. **Separate the API from node-level work.** The ClusterPolicy is the control plane API; the DaemonSets do the work on each node.
2. **Reconcile on every relevant change.** Node labels, owned DaemonSets, the custom resource, and configuration changes all trigger reconciliation.
3. **Report state with status and conditions.** Users need to know whether the desired version is deployed, available, degraded, or still being validated.
4. **Represent partial failure.** A driver can fail on one node while others succeed. Status must identify which components and nodes are unhealthy rather than report a single boolean.
5. **Use DaemonSets for per-node components,** with node selectors based on discovered labels.
6. **Own the configuration you generate.** If other people or tools edit a ConfigMap that the operator generates, the cluster drifts from the custom resource. The operator should overwrite such changes or read configuration only from its API.
7. **Validate with real workloads.** A validation Pod that uses the GPU catches failures that Pod readiness does not.
8. **Coordinate upgrades of coupled components.** The driver, toolkit, and device plugin versions depend on each other, so the operator upgrades them in a controlled order.
9. **Make failures observable.** Expose metrics and Kubernetes events for component state changes, driver failures, and allocation failures.
10. **Use leader election** so that only one controller replica changes cluster state at a time.

8. Troubleshooting

`nvidia.com/gpu` is missing from the node's allocatable resources

- Check that the device plugin Pod is running on the node: `kubectl get pods -n gpu-operator -o wide --field-selector spec.nodeName=<node>`.
- Read the device plugin logs for registration errors or a failure to load NVML (the NVIDIA Management Library), which usually means the driver is not ready.
- Check that the driver works on the host with `nvidia-smi`. If the driver is not loaded, the plugin cannot list any devices.
- If the kubelet restarted recently, check that the plugin registered again: `journalctl -u kubelet | grep -i "device plugin"`.
- Confirm that the node has a GPU. A CPU-only node is not expected to report `nvidia.com/gpu`.

A Pod is Pending with 0/N nodes are available

```
kubectl describe pod <pod>
kubectl describe node <node> | grep -A5 'Allocatable'
kubectl get nodes -l nvidia.com/gpu.present=true
```

Common causes:

- No node has a GPU product label that matches the Pod's node selector.
- The nodes with matching GPUs have no unallocated GPUs.
- The GPU nodes have a taint that the Pod does not tolerate.
- The Pod requests more GPUs than any single node has.
- With the mixed MIG strategy, the Pod requests `nvidia.com/gpu` but the nodes advertise MIG profile resources instead.

The Pod runs, but `nvidia-smi` fails inside the container

- Check that the runtime class exists: `kubectl get runtimeclass nvidia`.
- Read the container toolkit logs: `kubectl logs -n gpu-operator <toolkit-pod>`.
- Check whether the device environment was set: `kubectl exec <pod> -- env | grep NVIDIA`.
- Check that the host driver supports the CUDA version in the image. `nvidia-smi` on the host shows the highest CUDA version the driver supports.

GPU hardware errors

```
dmesg -T | grep -i xid
nvidia-smi -q -d ECC
nvidia-smi -q -d ROW_REMAPPER      # Ampere and newer
nvidia-smi -q -d PAGE_RETIREMENT   # older architectures
kubectl logs -n gpu-operator <dcgm-exporter-pod>
```

9. Interview questions

How does a Pod get a GPU?

The Pod requests `nvidia.com/gpu` in its limits. The scheduler places it on a node with enough allocatable GPUs. The kubelet's device manager selects specific GPUs and calls the device plugin's `Allocate` method, which returns the device IDs as environment variables or CDI device names. The container runtime uses them to add the device nodes and driver libraries to the container, and the application can use the GPU.

What does the device plugin do?

It registers the `nvidia.com/gpu` resource with the kubelet, reports the list of devices and their health through `ListAndWatch`, and, at allocation time, translates the kubelet's choice of devices into the settings the container runtime needs.

What is the difference between the device plugin and the container toolkit?

The device plugin integrates with Kubernetes: it tells the kubelet how many GPUs exist and which ones a container receives. The container toolkit integrates with the container runtime: it makes the assigned GPUs and the driver libraries visible inside the container. The GPU Operator deploys both.

How are drivers upgraded without disrupting workloads?

A driver cannot be replaced while processes use the GPU. The GPU Operator's upgrade controller upgrades a limited number of nodes at a time: it cordons each node, drains or waits for its GPU Pods, replaces the driver Pod, validates the node, and uncordons it. The policy is configured in the `ClusterPolicy`, including how many nodes may upgrade in parallel. Running containers are never modified in place; Kubernetes reschedules workloads around the nodes being upgraded.

Why are most components DaemonSets?

The driver, toolkit, device plugin, DCGM exporter, and MIG manager must each run exactly once on every GPU node. A DaemonSet runs one Pod on each node that matches its selector, starts a Pod automatically on new nodes, and supports rolling updates.

How would you apply the GPU Operator's design to another operator?

Define the desired state as a custom resource. Watch node labels and the resources you own. Use DaemonSets for per-node components. Report status with conditions and `observedGeneration`, including per-node failures. Validate with real workloads, and export operational metrics.

Summary

- A GPU node needs a driver, a configured container runtime, a device plugin, labels, monitoring, and validation. The GPU Operator installs and manages all of them from a ClusterPolicy.
- NFD and GPU Feature Discovery label nodes; the operator uses the labels to target DaemonSets at GPU nodes.
- The scheduler counts GPUs as integers. The kubelet and device plugin choose specific devices, and the container toolkit makes them visible in the container.
- MIG provides hardware isolation; time-slicing and MPS share a GPU with weaker isolation.
- The GPU Operator shows practices that apply to any operator: status that represents partial failure, DaemonSets for node-level work, validation with real workloads, and coordinated upgrades.

Source: [10-gpu-operator-kubebuilder-deep-dive.md](#) · [PDF](#) · [All study guides](#)